МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
Львівський національний університет імені Івана Франка
Факультет електроніки та комп’ютерних технологій

ЗВІТ
З ЛАБОРАТОРНОЇ РОБОТИ № 7

з дисципліни “Професійний Python”

на тему “ Робота з базами даних у Python “

Виконав
студент 3 курсу
групи ФЕП-32
Козачок Ростислав Михайлович

Перевірив
Куньо Іван Михайлович

Львів – 2026

Мета: Ознайомитися з основними принципами організації шару зберігання даних у Python-застосунках, навчитися працювати з реляційними базами даних, мовою SQL, СУБД SQLite, стандартом Python DB-API, параметризованими запитами, CRUD-операціями, транзакціями, принципами ACID, патерном Repository, SQLAlchemy ORM та міграціями бази даних.

Навчитися інтегрувати persistence layer у Python-проєкт, створений у попередніх лабораторних роботах, та перевіряти його за допомогою автоматизованих integration tests.

Варіант 14. Система керування завданнями

Хід роботи:

Структура проєкту (організація файлів за стандартним src-layout)

```
task_manager/
├── pyproject.toml
├── README.md
├── alembic.ini
├── migrations/
│   └── versions/001_add_task_deadline.py
├── src/
│   └── task_manager/
│       ├── database.py
│       ├── models.py
│       ├── repositories.py
│       ├── services.py
│       └── main.py
└── tests/
    ├── conftest.py
    └── test_repositories.py
```

Схема бази даних



Приклад параметризованого запиту (через Python DB-API / sqlite3) ORM-моделі (реалізація за допомогою SQLAlchemy)



```python
def filter_tasks_dbapi(
    database_path: str,
    status: str,
    priority: str,
) -> list[tuple]:
    """
    Демонстрація роботи з Python DB-API (sqlite3) та параметризованим запитом.
    Повертає завдання за status та priority із JOIN таблиці users.
    """
    with sqlite3.connect(database_path) as connection:
        cursor = connection.cursor()
        cursor.execute(
            """
            SELECT
                tasks.id,
                tasks.title,
                tasks.priority,
                tasks.status,
                users.username
            FROM tasks
            LEFT JOIN users ON tasks.user_id = users.id
            WHERE tasks.status = ? AND tasks.priority = ?
            ORDER BY tasks.id ASC
            """,
            (status, priority),
        )
        return cursor.fetchall()
```

Код файлу database.py (налагодження engine, SessionLocal та pragma для FOREIGN KEY)



```python
from typing import Any
from sqlalchemy import create_engine, event
from sqlalchemy.engine import Engine
from sqlalchemy.orm import sessionmaker

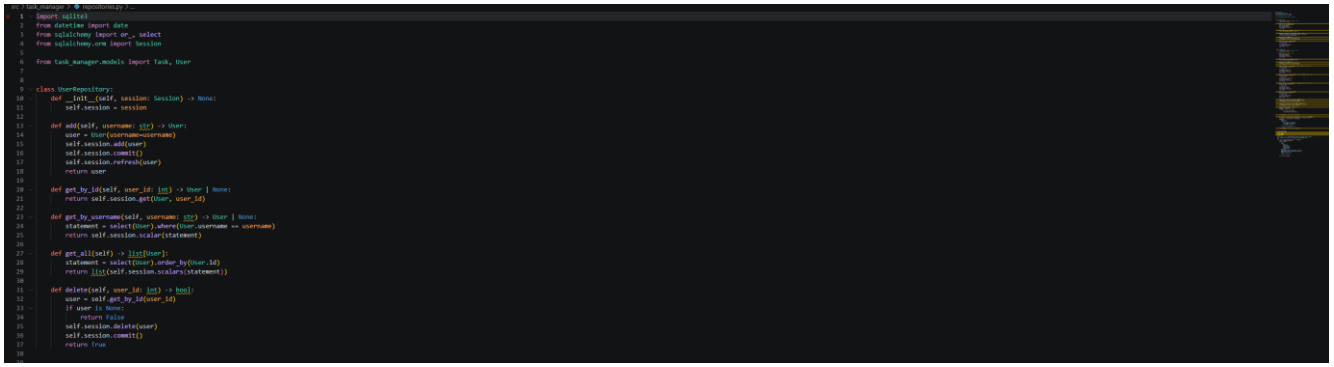
DATABASE_URL = "sqlite:///tasks.db"

engine = create_engine(
    DATABASE_URL,
    echo=False,
)


@event.listens_for(engine, "connect")
def set_sqlite_pragma(dbapi_connection: Any, connection_record: Any) -> None:
    """Увімкнення підтримки FOREIGN KEY у SQLite."""
    cursor = dbapi_connection.cursor()
    cursor.execute("PRAGMA foreign_keys=ON")
    cursor.close()


SessionLocal = sessionmaker(
    bind=engine,
    autoflush=False,
    expire_on_commit=False,
)
```

Реалізація Repository Pattern (UserRepository та TaskRepository)



```python
import sqlite3
from datetime import date
from sqlalchemy import or_, select
from sqlalchemy.orm import Session

from task_manager.models import Task, User


class UserRepository:
    def __init__(self, session: Session) -> None:
        self.session = session

    def add(self, username: str) -> User:
        user = User(username=username)
        self.session.add(user)
        self.session.commit()
        self.session.refresh(user)
        return user

    def get_by_id(self, user_id: int) -> User | None:
        return self.session.get(User, user_id)

    def get_by_username(self, username: str) -> User | None:
        statement = select(User).where(User.username == username)
        return self.session.scalar(statement)

    def get_all(self) -> list[User]:
        statement = select(User).order_by(User.id)
        return list(self.session.scalars(statement))

    def delete(self, user_id: int) -> bool:
        user = self.get_by_id(user_id)
        if user is None:
            return False
        self.session.delete(user)
        self.session.commit()
        return True
```



```python
class TaskRepository:
    def __init__(self, session: Session) -> None:
        self.session = session

    def add(self, task: Task) -> Task:
        self.session.add(task)
        self.session.commit()
        self.session.refresh(task)
        return task

    def get_by_id(self, task_id: int) -> Task | None:
        return self.session.get(Task, task_id)

    def get_all(self) -> list[Task]:
        statement = select(Task).order_by(Task.id)
        return list(self.session.scalars(statement))

    def update_status(self, task_id: int, new_status: str) -> Task | None:
        task = self.get_by_id(task_id)
        if task is None:
            return None
        task.status = new_status
        self.session.commit()
        self.session.refresh(task)
        return task

    def update_priority(self, task_id: int, new_priority: str) -> Task | None:
        task = self.get_by_id(task_id)
        if task is None:
            return None
        task.priority = new_priority
        self.session.commit()
        self.session.refresh(task)
        return task

    def assign_user(self, task_id: int, user_id: int | None) -> Task | None:
        task = self.get_by_id(task_id)
        if task is None:
            return None
        task.user_id = user_id
        self.session.commit()
        self.session.refresh(task)
        return task

    def delete(self, task_id: int) -> bool:
        task = self.get_by_id(task_id)
        if task is None:
            return False
        self.session.delete(task)
        self.session.commit()
        return True

    def filter_by_status(self, status: str) -> list[Task]:
        statement = select(Task).where(Task.status == status)
        return list(self.session.scalars(statement))

    def filter_by_priority(self, priority: str) -> list[Task]:
        statement = select(Task).where(Task.priority == priority)
        return list(self.session.scalars(statement))

    def search(self, query: str) -> list[Task]:
        pattern = f"%{query}%"
        statement = select(Task).where(
            or_(
                Task.title.like(pattern),
                Task.description.like(pattern),
```

Приклади CRUD-операцій (Create, Read, Update, Delete)



```python
def test_crud_user_and_task(session: Session) -> None:
    user_repo = UserRepository(session)
    task_repo = TaskRepository(session)

    user = user_repo.add("ivan_dev")
    assert user.id is not None

    task = task_repo.add(
        Task(
            title="Fix bug #42",
            description="Resolve memory leak",
            priority="high",
            status="todo",
            user_id=user.id,
        )
    )
    assert task.id is not None

    loaded = task_repo.get_by_id(task.id)
    assert loaded is not None
    assert loaded.title == "Fix bug #42"
    assert loaded.user_id == user.id

    updated_status = task_repo.update_status(task.id, "in_progress")
    assert updated_status is not None
    assert updated_status.status == "in_progress"

    updated_prio = task_repo.update_priority(task.id, "low")
    assert updated_prio is not None
    assert updated_prio.priority == "low"

    deleted = task_repo.delete(task.id)
    assert deleted is True
    assert task_repo.get_by_id(task.id) is None
```

Приклад транзакції (пакетне перепризначення завдань та зміна статусу)



```python
def batch_reassign_and_update_status(
    session: Session,
    task_ids: list[int],
    new_user_id: int,
    new_status: str,
) -> None:
    """
    Транзакційна операція: пакетне перепризначення користувача
    та зміна статусу для списку задач. Якщо будь-яка задача не існує,
    користувач не знайдений або статус некоректний — виконується rollback.
    """
    try:
        if new_status not in VALID_STATUSES:
            raise ValueError(f"Invalid status: {new_status}")

        user = session.get(User, new_user_id)
        if user is None:
            raise ValueError(f"User with id={new_user_id} not found")

        for task_id in task_ids:
            task = session.get(Task, task_id)
            if task is None:
                raise ValueError(f"Task with id={task_id} not found")

            task.user_id = new_user_id
            task.status = new_status

        session.commit()
    except Exception:
        session.rollback()
        raise
```

Пояснення commit і rollback

commit() - фіксує та записує зміни у файл БД постійно;
rollback() - скасовує проміжні зміни транзакції у разі виникнення помилки, повертаючи базу у вихідний стан.

Пояснення принципів ACID (Atomicity, Consistency, Isolation, Durability)

- **Atomicity (Атомарність):** Пакетне перепризначення виконується повністю або скасовується повністю.
- **Consistency (Узгодженість):** Дотримання обмежень FOREIGN KEY, UNIQUE та CHECK (для статусу й пріоритету).
- **Isolation (Ізольованість):** Паралельні транзакції не псують дані одна одної.
- **Durability (Довговічність):** Після commit() дані надійно зберігаються на диску.

Міграція бази даних (використання Alembic та додавання поля deadline)



```python
from typing import Sequence, Union
from alembic import op
import sqlalchemy as sa

revision: str = "001_add_deadline"
down_revision: Union[str, None] = None
branch_labels: Union[str, Sequence[str], None] = None
depends_on: Union[str, Sequence[str], None] = None


def upgrade() -> None:
    with op.batch_alter_table("tasks", schema=None) as batch_op:
        batch_op.add_column(
            sa.Column("deadline", sa.Date(), nullable=True)
        )


def downgrade() -> None:
    with op.batch_alter_table("tasks", schema=None) as batch_op:
        batch_op.drop_column("deadline")
```

Код інтеграційних тестів (conftest.py з тестовою БД у пам'яті та тести у test_repositories.py)



```python
from typing import Any, Generator
import pytest
from sqlalchemy import create_engine, event
from sqlalchemy.orm import Session, sessionmaker

from task_manager.models import Base


@pytest.fixture
def session() -> Generator[Session, None, None]:
    engine = create_engine("sqlite:///:memory:")

    @event.listens_for(engine, "connect")
    def _enable_fk(dbapi_connection: Any, connection_record: Any) -> None:
        cursor = dbapi_connection.cursor()
        cursor.execute("PRAGMA foreign_keys=ON")
        cursor.close()

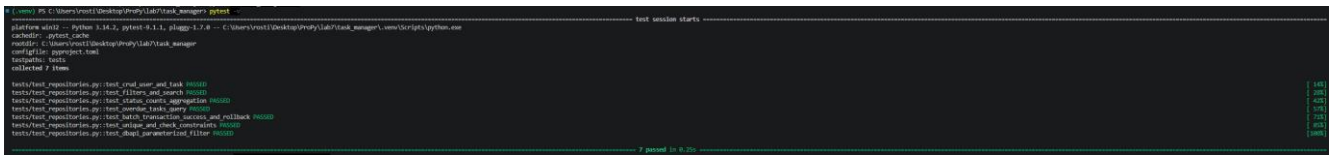
    Base.metadata.create_all(engine)

    TestSession = sessionmaker(
        bind=engine,
        autoflush=False,
        expire_on_commit=False,
    )

    with TestSession() as test_session:
        yield test_session

    Base.metadata.drop_all(engine)
```

Результат виконання тестів (pytest -v)



Скріншоти роботи програми (вивід у консолі)



Висновок:

У ході виконання лабораторної роботи №7 було розроблено та інтегровано повноцінний шар зберігання даних (persistence layer) для системи керування завданнями (Варіант 14). Було спроєктовано реляційну схему БД у SQLite із двома пов'язаними таблицями (users та tasks), налаштовано первинні й зовнішні ключі, а також обмеження цілісності (UNIQUE, CHECK). За допомогою SQLAlchemy 2.0 створено декларативні ORM-моделі та реалізовано патерн Repository (UserRepository, TaskRepository), що дозволило повністю відокремити бізнес-логіку (TaskService) від деталей доступу до бази даних. Окремо продемонстровано роботу зі стандартним інтерфейсом Python DB-API (sqlite3) із використанням безпечних параметризованих запитів, реалізовано транзакційне пакетне оновлення задач із дотриманням принципів ACID (commit/rollback), виконано міграцію схеми через Alembic для додавання поля deadline та перевірено всі компоненти інтеграційними тестами pytest на ізольованій базі даних у пам'яті.